Data Science Collective

Member-only story

Artificial Intelligence + | Data Science ✓ | Machine Learning ✓ | Programming ✓

Software Development +

Jev Explained: TypeSafe's AI Model for Workflow Automation

From RLHF to RLCD: how TypeSafe's System One Models use fast, probabilistic decisions to bring AI deeper into software workflows.



Shreyansh Jain Follow 15 min read · Sep 19, 2026

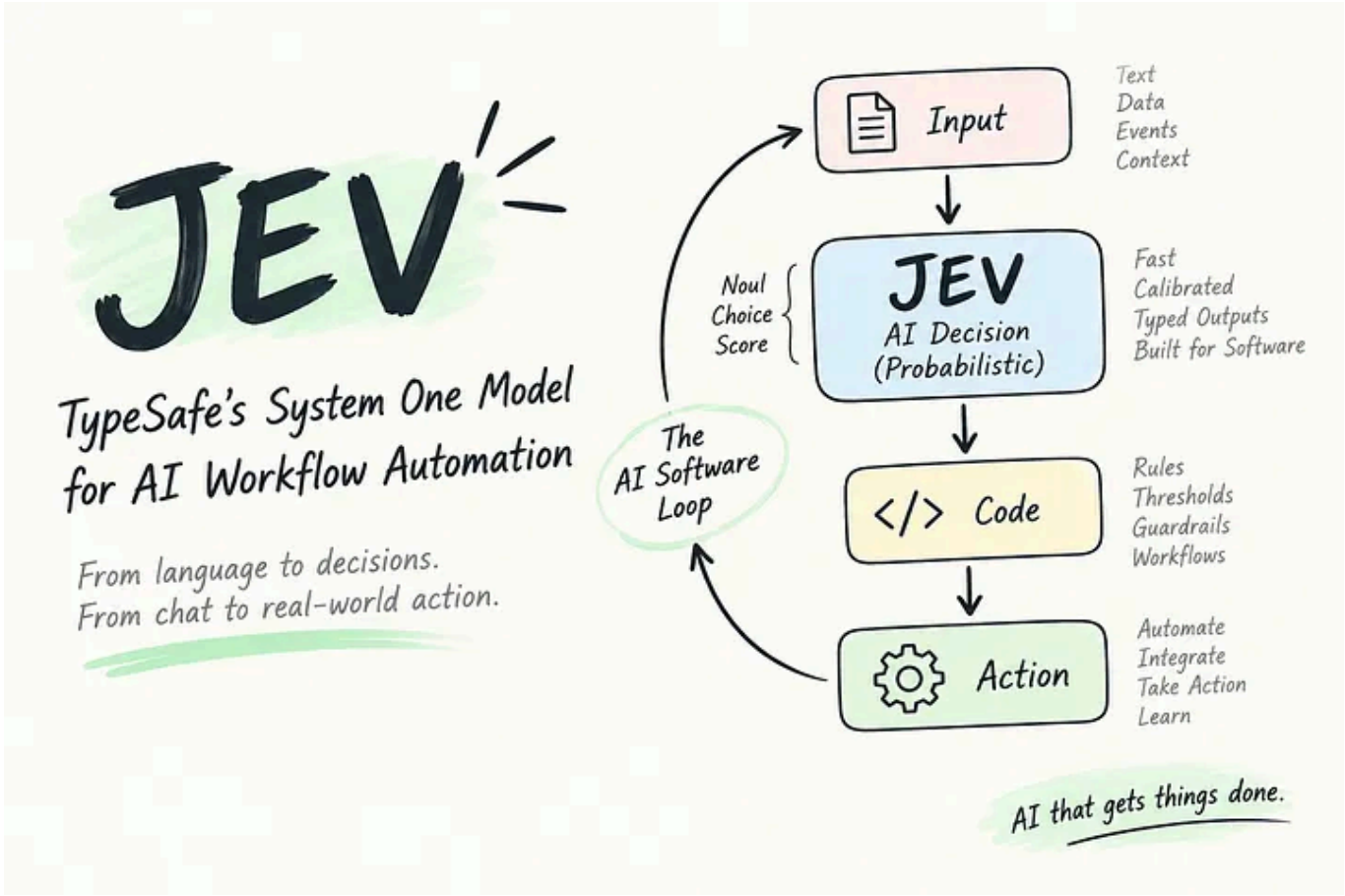
223 1

Claude Code, Codex, and all the other Frontier AI tools focus on coding assistance, human conversation and reasoning, but they still lack something

important: **workflow automation.**

And this is what Jev focuses on.

> No Membership? No worries , read it for free here -> LINK



Diogo Almeida is the founder of ***typesafe.ai***, the person behind RLHF research that helped shape modern instruction-following models, and a contributor to the development of GPT-era systems. Now he is working on Jev — a model that TypeSafe claims can be **40x–200x faster**, with effectively free output tokens and significantly lower input-token costs for the workloads it targets.

Jev is being described under a new category called **System One Models.**

> *System 1, huh? Sounds like something new, right?*

TL;DR on Jev

Matija Sosic shared a cool 45 sec demo on his twitter handle and this was the best i could find. Yet i'll encourage you to give this article a read to understand what Jev actually is and where are we heading. In the end i have attached some cool usecases of Jev i could find.

Matija Sosic
@MatijaSosic · **Follow**

Here's a 45-second TL;DR on Jev.

I find the core idea beautifully simple, but the video made it really hard to understand.

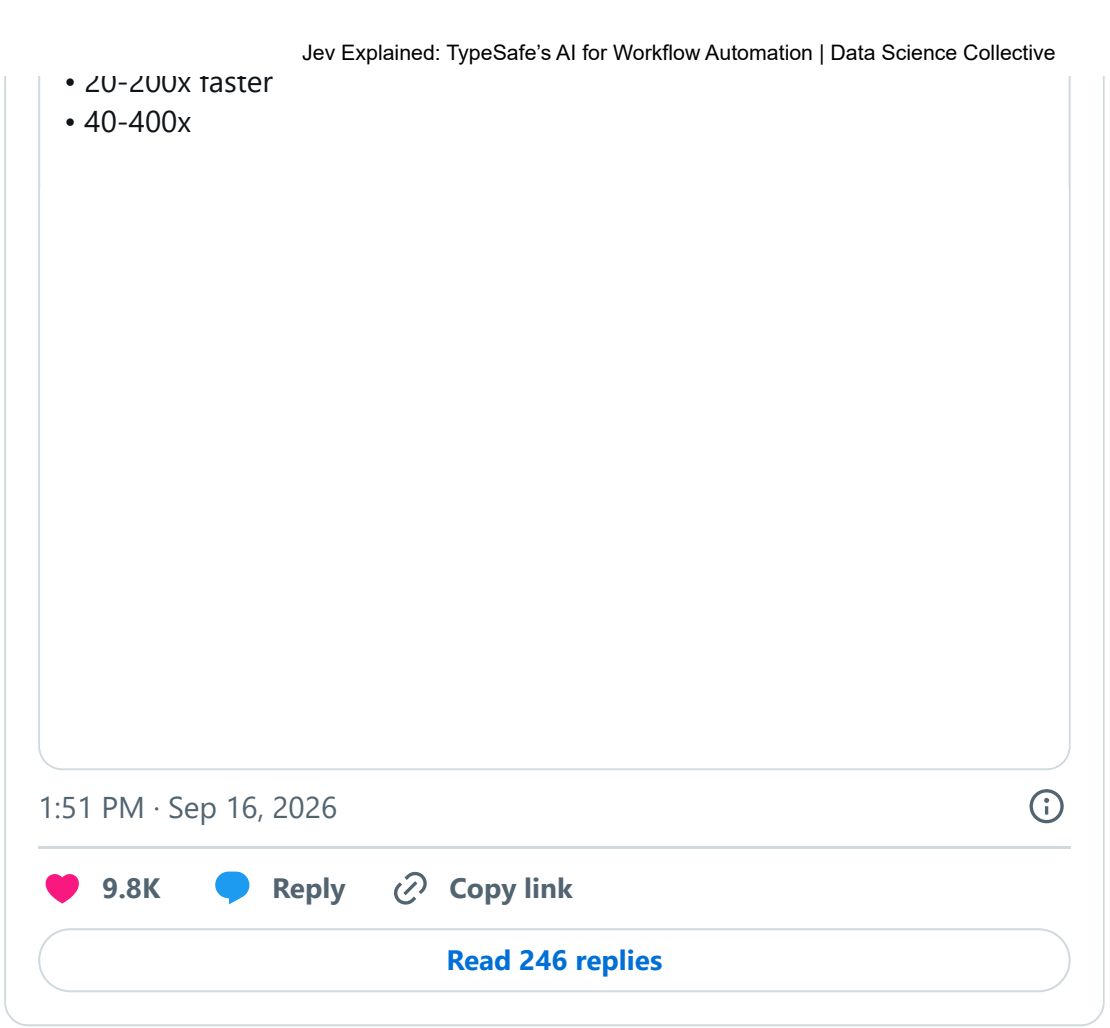
Hope you find it helpful.

Watch on X

Diogo Almeida @CompleteSkeptic

After co-inventing ChatGPT, I kept asking myself: why have superhuman chat models not led to AGI?

I've spent the last 2 years in stealth building a new way to train models (RLCD), and a new type of frontier AI model that we are releasing today: Jev



Thinking, Fast and Slow

Actually, **not really.**

Thinking, Fast and Slow is a book published back in 2011 by Daniel Kahneman, the Israeli-American psychologist, where he describes two distinct modes of human cognition: **System 1 and System 2.**

System 1 is the fast, automatic and intuitive part of our thinking, whereas System 2 is the slower, more deliberate part.

TypeSafe borrows this terminology to describe a different kind of AI system.

While today's frontier models such as Claude and GPT are primarily built around language generation and increasingly sophisticated reasoning, Jev is designed around **fast, structured decision-making inside software.**

And this is where things start getting interesting. Jev isn't simply trying to be a smaller or faster version of GPT. It is trying to approach AI from a completely different abstraction.

This brings me back to something much older than LLMs:

Logic Gates

The image below shows the layer of abstraction that most of us would have encountered in Computer Science 101.

Logic gates and registers are what I want you to focus on. If you remember, they work on certain defined conditions and rules , essentially taking inputs, processing them, and producing an output. And Jev follows a somewhat similar philosophy. But there is one important difference. Traditional logic gates are **deterministic**. Real-world decisions aren't.

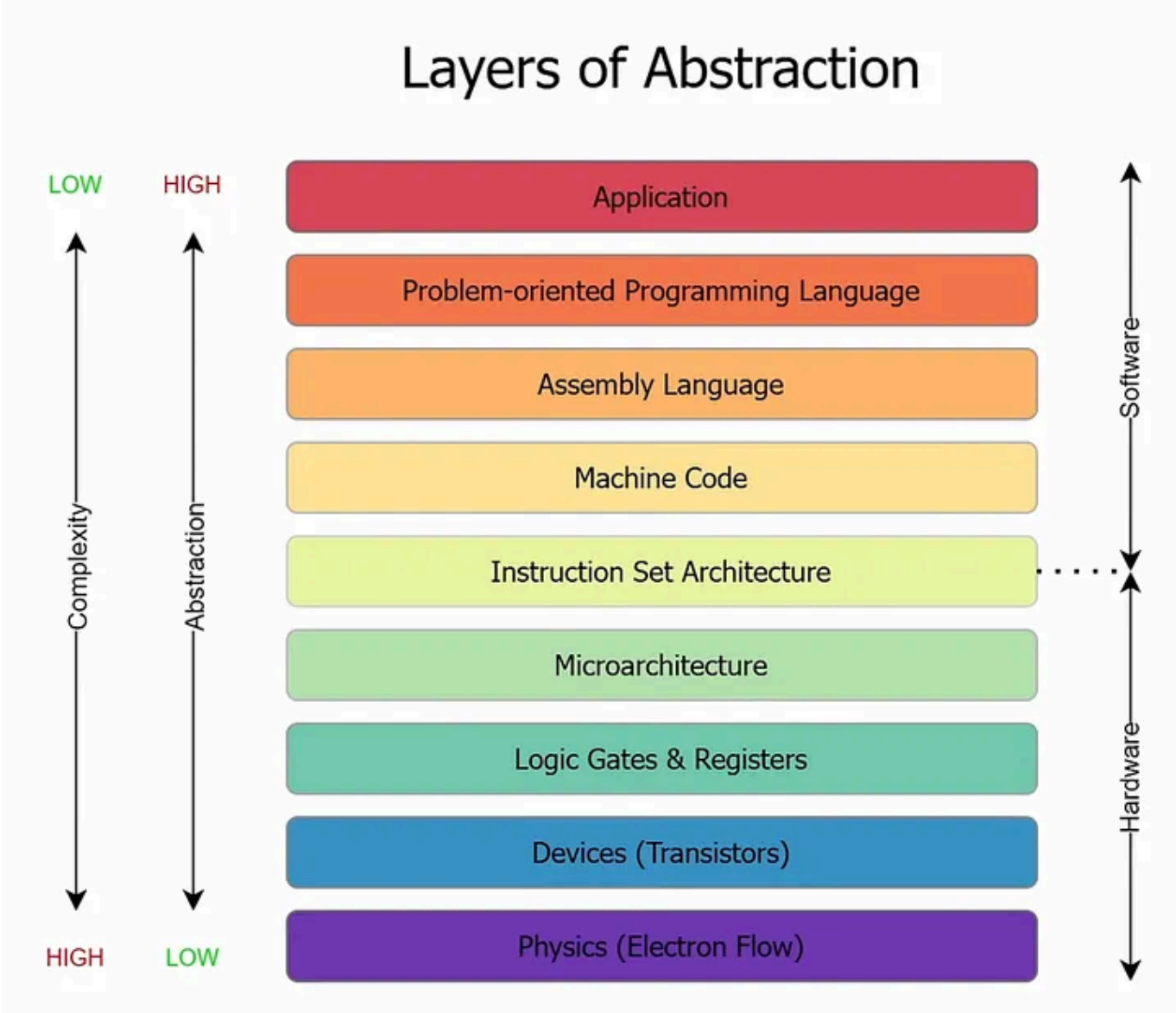
If I write:

```
IF A AND B
    → TRUE
```

there is no uncertainty involved. But imagine asking:

> Should this customer be escalated to a human?

You can't simply write a rule that perfectly captures frustration, intent, risk, urgency, context, and everything else contained inside a human conversation.



This is where Jev comes in.

Instead of asking an LLM to generate a paragraph and then asking your software to interpret that paragraph, Jev produces **structured decisions with probabilities.**

For example:

```
Is the customer frustrated?
→ YES — 94%
Is this a billing issue?
→ YES - 91%
Should this be escalated?
→ YES - 93%
```

And this is an important distinction:

Jev provides the intelligent judgment. Your software decides what to do with that judgment.

So you could have something as simple as:

```
IF escalation_probability > 90%
    → Escalate to human
ELSE
    → Continue automated workflow
```

But Why a New Training Approach?

How do you actually train a model to behave this way? To understand why TypeSafe created something called **RLCD**, we need to go all the way back to the technique that helped turn raw language models into the chat models we use every day. Yep.

RLHF.

RLHF: Teaching a Model What Humans Like

At its core, a language model learns to predict what comes next. Give it:

```
The capital of France is
```

and it learns that the next token is probably:

```
Paris
```

Do this at absurd scale, across absurd amounts of data, and you get a model that is extremely good at generating language. But there is a problem. **Being good at predicting text is not the same thing as being good at following instructions.** A base model could answer a question in a way that is technically plausible but overly verbose, unhelpful, toxic, weirdly formatted, or simply not what the user wanted. So OpenAI and others started optimizing these models with additional training signals.

One of the most influential approaches became **Reinforcement Learning from Human Feedback — RLHF**. The easiest way to understand RLHF is as a three-part pipeline.

Step 1: Show the Model Good Examples

First, humans demonstrate the behavior we want. For example:

```
User:
Explain quantum computing to a 10-year-old.
```

```
Human-written answer:
Imagine a normal computer bit as a light switch...
```

These demonstrations are used for **supervised fine-tuning.** The model is no longer being trained only to continue text. It is being trained to follow instructions. This was one of the core steps in the InstructGPT work.

Step 2: Ask Humans Which Answer They Prefer

Now imagine giving the model a prompt and asking it to produce two answers.

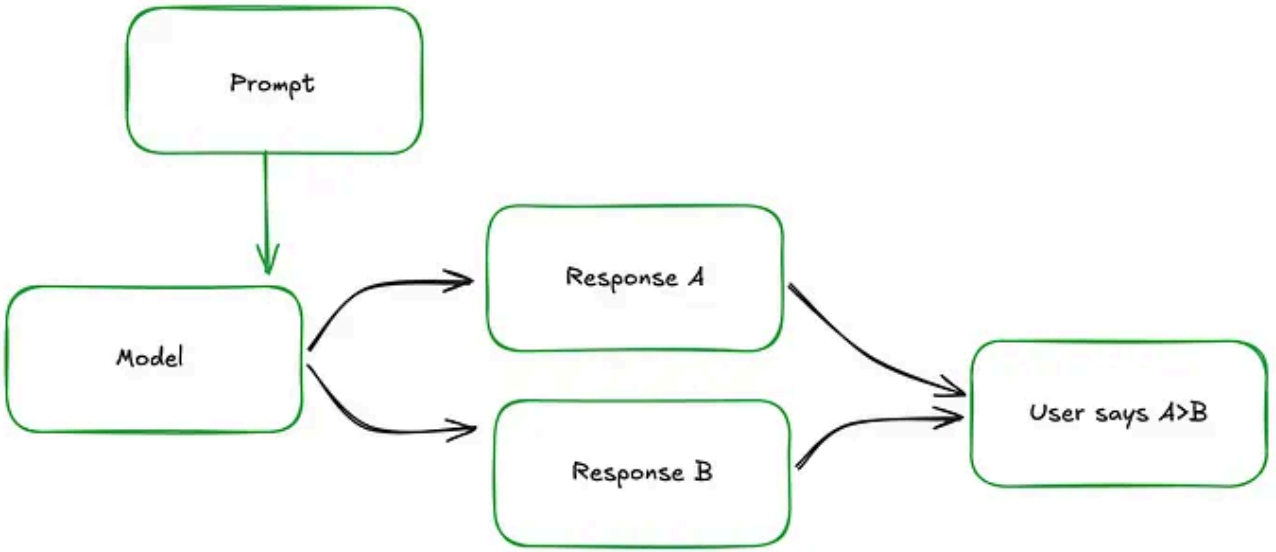
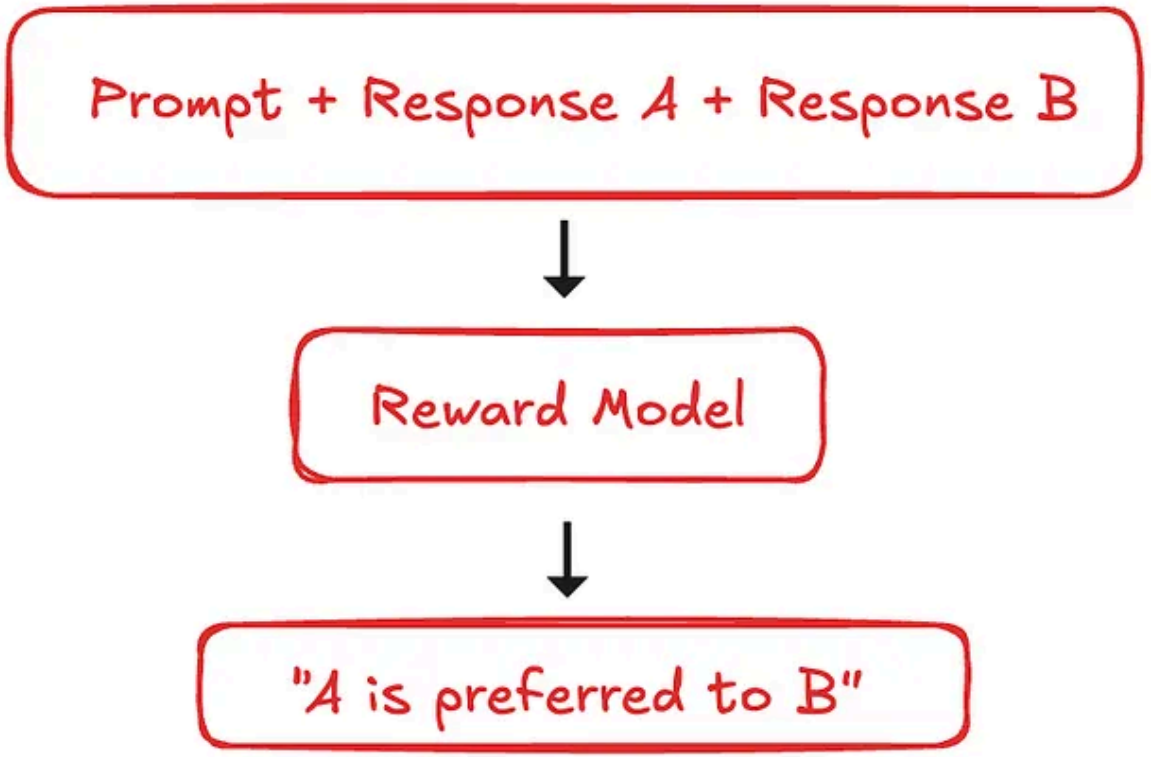


Image by Author on Human preference

Do this again. And again. And again. Eventually, you have a big dataset of human preferences. Notice what happened here. Humans didn't have to write the perfect answer anymore. They just had to say **which answer they preferred**. That turns a vague idea like "this response feels better" into a training signal.

Step 3: Train a Reward Model

But there is an obvious problem. You can't hire an army of humans to sit around judging every single response the model will ever generate. So instead, you train another model. A **Reward Model** learns to predict the human preference signal. Conceptually:



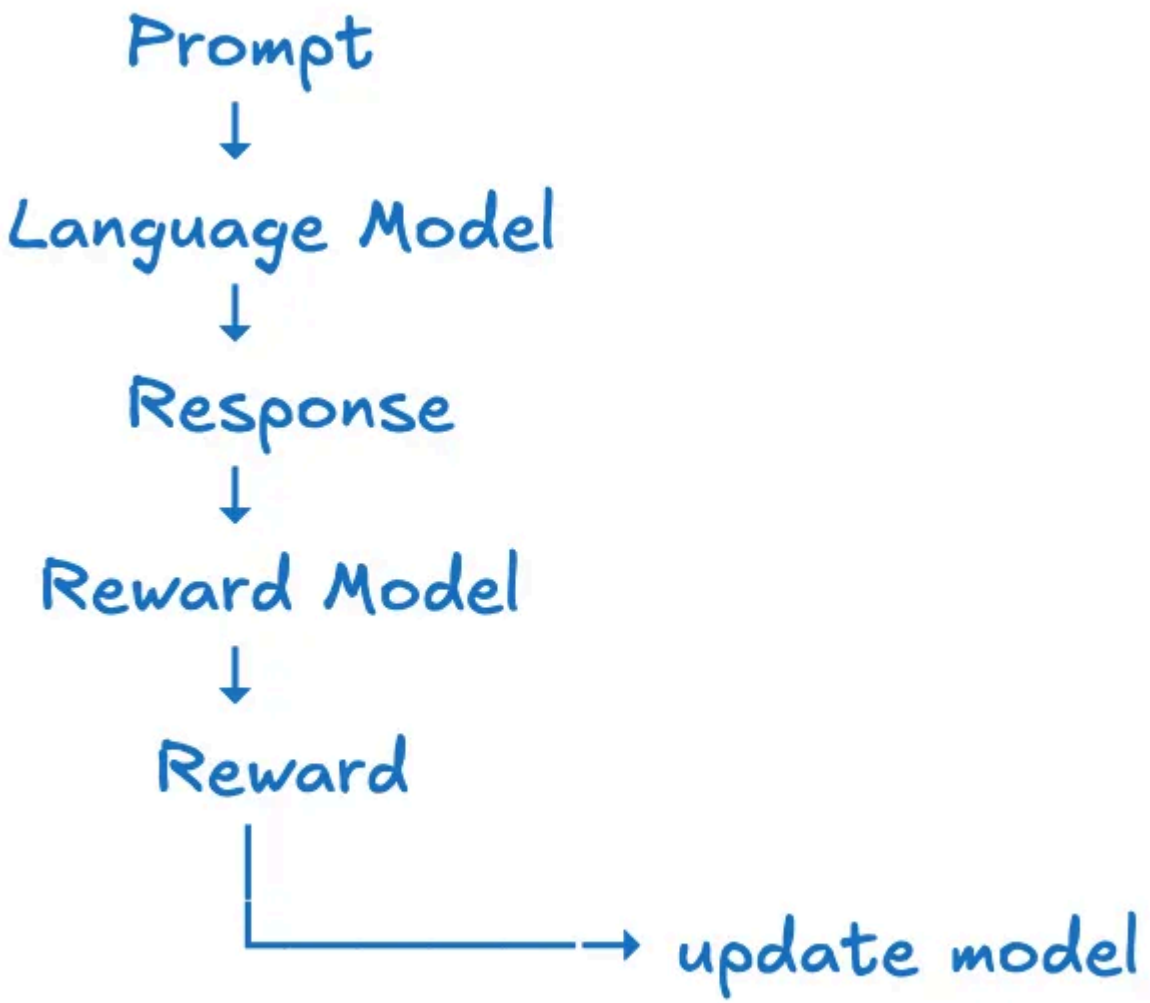
Now we can use that reward model as an automated judge.

Step 4: Actually Do the Reinforcement Learning

Now the language model generates responses. The reward model scores them. Higher reward means *"Cool. Do more things like this."* Lower reward

means *"Maybe don't do that again."*

The model is then updated to increase the expected reward. The original InstructGPT work used **PPO**, a reinforcement-learning algorithm, for this optimization. So the simplified loop is:


Prompt
Language Model
Response
Reward Model
Reward
update model

And this was a huge deal. The model wasn't merely learning to predict text anymore. It was being optimized toward **what humans considered a better response**. That idea became a major part of the recipe behind modern instruction-following assistants. And this is the important bit for our story:

> **RLHF optimized AI around a human preference signal.**

That is fantastic when a human is going to read the output. But TypeSafe is asking a different question.

What Happens When the Human Disappears?

Imagine the model is no longer writing a response for you. Instead, it is sitting quietly inside a piece of software.
A customer-support system asks: *Should I escalate this ticket?*

An invoice system asks: *Should I pay this invoice?*

A security system asks: *Should I isolate this machine?*

A browser agent asks: *Which action should I take next?*

These aren't really **conversation problems.** They are **decision problems.** And there is a subtle but massive difference between the two.

If I ask an LLM: *"Write me a response to this customer."*

I care about the quality of the response.

But if I ask: *"Should I escalate this customer?"*

I don't need a beautiful paragraph. I need something software can use. Something like:

```
YES
Probability: 0.93
```

Because now I can write:

```
if escalation_probability > 0.90:
```

```
        escalate()
```

And suddenly that little number **0.93,** matters a lot. Which raises the obvious question: ***What does the 93% actually mean?***

And that brings us to one of the most important concepts in this entire article.

Calibration

The word sounds boring. It really isn't. Imagine a weather model that tells you: *"There is a 70% chance of rain."*

Okay. Now imagine collecting **100 predictions** from that model where it said 70%. If the model is well calibrated, you would expect rain on roughly **70 of those 100 occasions.** Not necessarily 70 specific predictions getting individually correct. What matters is that, over many predictions, the model's confidence lines up with reality. That is the basic idea of **calibration.**

Accuracy Is Not Calibration

This distinction is easy to miss. Accuracy asks: ***"Was I right?"***

Calibration asks: *"When I said 80%, did reality look roughly like 80%?"*

For example:

```
100 predictions at 70% confidence
→ 70 correct
→ roughly calibrated

100 predictions at 70% confidence
→ 50 correct
→ overconfident

100 predictions at 70% confidence
→ 90 correct
→ underconfident
```

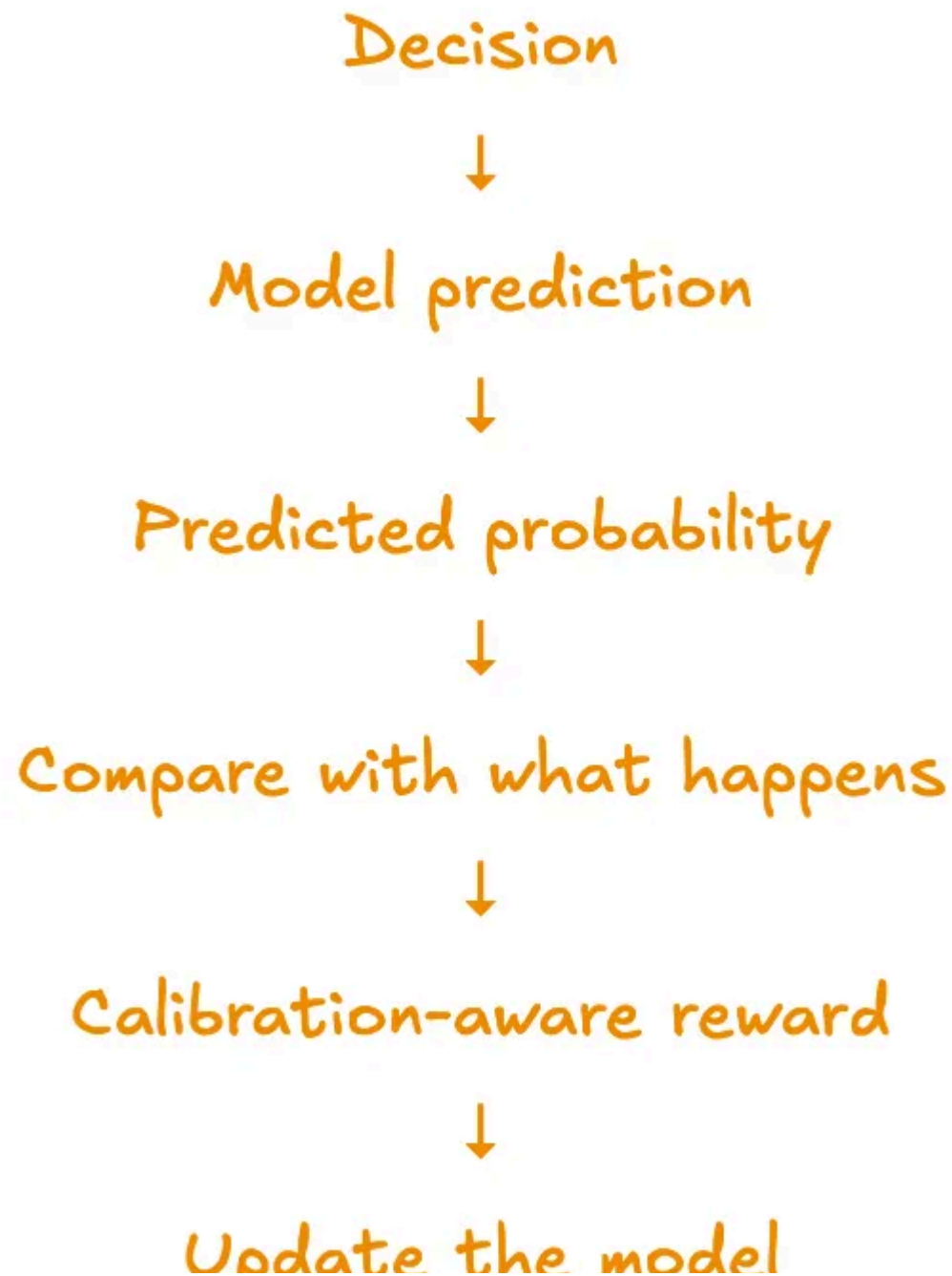
The important part is that confidence only becomes meaningful when we compare it with reality. And that matters enormously when software is going to act on the number.

A human can read an answer and think "Hmm, this model sounds uncertain."

Code can't. Code sees a number.

So What Is RLCD?

Now the name **Reinforcement Learning for Calibrated Decisions** starts to make a lot more sense. TypeSafe describes RLCD as the training approach behind Jev, with the goal of producing decisions whose probabilities are meaningfully calibrated. At a high level, you can think of the flow like this:


Decision
Model prediction
Predicted probability
Compare with what happens
Calibration-aware reward
Update the model

RLCD Flow by Author

The important part is the objective. RLHF asks, **"Which answer would a human prefer?"** RLVR asks, **"Can I verify whether this answer is correct?"** RLCD is aimed at a different question: **"Does the model's confidence correspond to reality?"**

So if a model repeatedly says **80%**, we want roughly 80% of comparable cases to actually turn out that way. The model is not just learning to pick an answer; it is learning to make the probability attached to that answer useful.

And that is about as far as we can responsibly go when talking about Jev specifically. TypeSafe has publicly described RLCD and its calibration objective, but it has **not published the complete RLCD training recipe or explained exactly how each part of that method is implemented inside Jev.** We don't know the exact reward function, loss, dataset construction, calibration procedure, or full optimization pipeline. So I won't pretend we do. For the rest of this article, I'll stick to what TypeSafe has actually disclosed about Jev and keep the internals of RLCD in the "interesting things we don't know yet" box.

One Small but Very Important Correction

There is a phrase around Jev that is easy to misunderstand: **"Zero hallucinations."** TypeSafe uses this phrase to describe Jev's type-safe interface and the fact that it does not generate arbitrary strings. That does **not** mean:

> "Jev can never make a wrong decision."

Of course it can. Suppose Jev is only allowed to answer:

```
YES
NO
```

It can still answer **YES when the real answer is NO**. What it cannot do is suddenly return:

```
Maybe-ish, but also here is a 700-word poem about invoices.
```

Medium Search Play Games Write

The output space is constrained. So the more precise way to think about it is:

> *Jev can eliminate invalid output types and arbitrary text-generation failures, but it can still make incorrect judgments.*

And that makes calibration even more important. The goal isn't to make the model infallible. It is to give software a useful signal for when it should act automatically and when it should ask for help.

Bring the Logic Gates Back

Remember the logic-gate analogy from the beginning? This is where it clicks. A traditional logic gate is deterministic:

```
A AND B
   ↓
TRUE
```

Jev is trying to introduce something different into software: **probabilistic intelligence primitives.** For example:

```
Is this suspicious?
→ YES
→ 0.91
```

Or:

```
Which route should this ticket take?
```

```
Billing       0.72
Technical     0.18
Sales         0.06
Other         0.04
```

Or a score:

```
Customer risk
→ 8.2 / 10
```

TypeSafe's workflow evaluation framework formalizes these primitives as **Noul, Choice and Score**.

- **Noul:** a yes/no question with a probability.
- **Choice:** choose among a fixed set of options.
- **Score:** produce a value on a defined scale.

The important part is not the names. The important part is the shape of the interface.

```
Unstructured State
       ↓
      Jev
       ↓
Typed decision + probability
       ↓
      Code
       ↓
     Action
```

That starts to look much less like a chatbot. And much more like a **new kind of software primitive**.

Why Not Just Use JSON Mode?

Someone will inevitably ask this. And it is a good question. A normal LLM can already return:

```
{
  "is_urgent": true,
  "category": "billing"
}
```

So… why do we need Jev? Because the fact that the output is JSON doesn't change what happened underneath. The model still generated a sequence of tokens. Something like:

```
{
  → "is
  → _urgent
  → "
  → :
  → true
  → }
```

Then your software parses the string, validates the JSON, checks the fields and finally uses the values. Jev is taking the opposite approach. Its output space is defined ahead of time. You are not asking *"Please write me JSON that represents your decision."* You are asking *"Here is the state. Here are the decisions I need."*

That difference sounds small. Architecturally, it is not. TypeSafe describes Jev as receiving unstructured state plus typed questions and returning typed probabilistic decisions. That is a much tighter interface between intelligence and software.

Okay, But Why Is It So Fast?

Now we finally get to the part everybody screenshots. **Jev is ridiculously fast.** TypeSafe's launch materials report Jev answering decisions in roughly **70–500 ms** end-to-end, while the company's workflow benchmark currently reports **193.6× faster and 444.6× cheaper** than the comparison frontier models it evaluated. And yes, those are the company's own numbers on its own evaluation setup, so treat them as vendor-reported benchmark results — not universal laws of physics. Still, the architectural reason for the speed is interesting.

Traditional LLMs Generate Sequentially

A standard autoregressive model generates one token at a time.



Sequential token generation — Image by Author

Each next token depends on what came before it. This is fantastic for writing. It is also a pretty expensive way to answer:

> "Is this suspicious?"

Jev gives up arbitrary text generation entirely. Instead, the questions and their answer spaces are known in advance, and TypeSafe says Jev uses a **parallel sampler** to produce the requested structured outputs in a single query. So the mental model becomes:

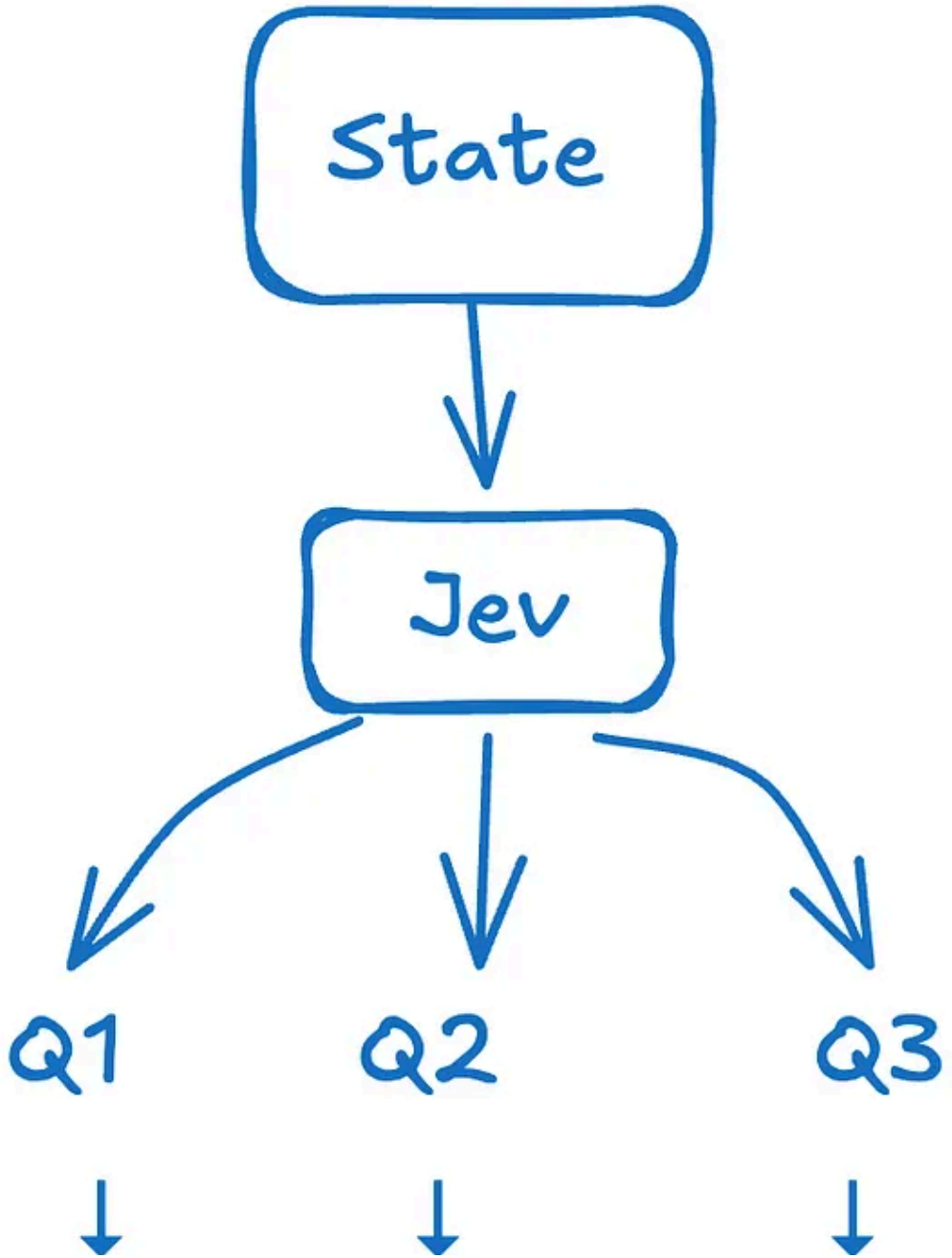

State
Jev
Q1
Q2
Q3



Image by Author — Mental model for Jev

You are not asking the model to write an essay about its decision. You are asking it to answer the decision directly. And there is an important design tradeoff hiding here:

> **Jev becomes dramatically cheaper partly because it refuses to do something expensive: generate arbitrary text.**

That is not a weakness if your task never needed the text in the first place. If I need an email, I still want an LLM. If I need a yes/no decision over a fixed action space, generating 700 words to arrive at "NO" feels a little like hiring a law firm to decide whether the office door is open.

But for AI applications, the first thing that scales horizontally is the **application layer**. More users, more agents, more workflows → more model calls. And if every little decision requires a full autoregressive LLM call, the cost starts multiplying very quickly.

There is another interesting problem with autoregressive models. The model generates one token, then uses that token as part of the context for generating the next one.

So if an incorrect token gets generated somewhere in the middle, that token becomes part of the context for everything that follows. It doesn't guarantee that every later token will be wrong, but **one bad prediction can send the rest of the generation down the wrong path**. The attention mechanism keeps using the generated context to predict what comes next, so an earlier mistake can influence the entire continuation.

And this is where **workflow automation** becomes a very different problem from chat.

Today's LLMs are incredibly good at talking to humans, writing code and generating text. But a workflow doesn't really care whether the AI can write a beautiful paragraph. It wants things like:

```
Is this invoice valid?
→ YES

Should this ticket be escalated?
→ NO

Which tool should I call?
→ refund_customer

Is the confidence high enough to act?
→ 94%
```

These are small, repeated decisions surrounded by deterministic rules and tool calls. Making an entire language model generate tokens for every one of them can be a rather expensive way of asking a very small question.

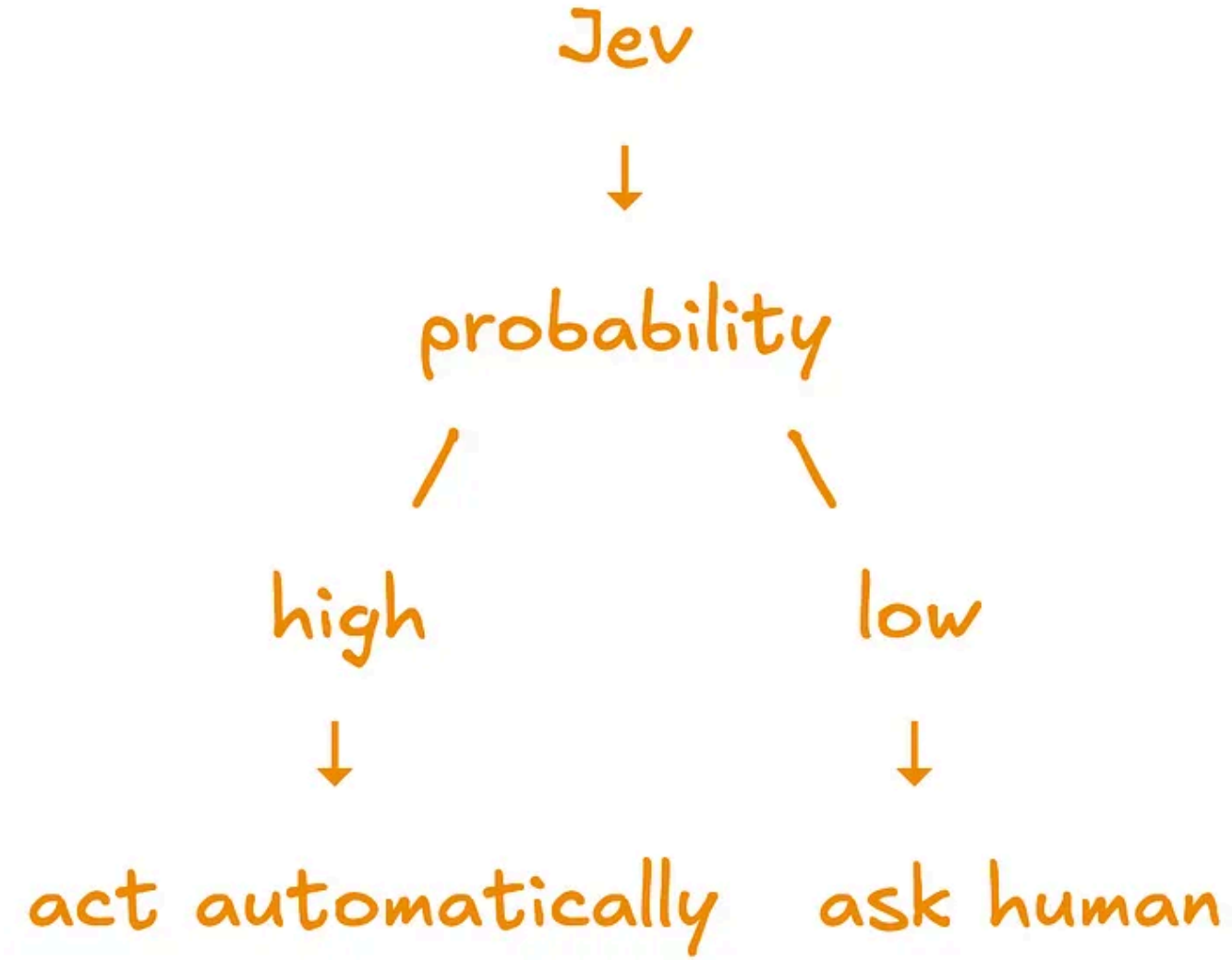
And this is the problem TypeSafe is trying to attack: not making AI better at talking to humans, but making AI cheaper, faster and more natural to use inside software workflows.

The Human Might Actually Become the Exception Handler

This is perhaps my favorite consequence of the whole idea. Today we often build AI systems like this:



The human is sitting in the middle of the loop because we don't fully trust the model. But if a model can provide useful calibrated probabilities, we can build something more like:



Now the human is no longer the default processor. The human becomes the **exception handler.** That is a pretty profound change in how software could

be designed. And importantly, this isn't "AI gets to do whatever it wants now." It is the opposite. Your code still sets the rules:

```
IF confidence > 0.95
    AND action is reversible
    AND amount < $100
        → execute
ELSE
        → human review
```

The model provides the intelligence. The system still provides the guardrails.

But There Is a Catch

There is always a catch. A calibrated model is not a magical oracle. Calibration is a **statistical property**. A prediction can still be wrong. And calibration can deteriorate when the world changes. Imagine you trained your system on one kind of customer behavior and six months later the product changed completely. A beautifully calibrated model from six months ago can become badly calibrated under the new distribution. So if you are building real automation, you still need:

```
monitoring
+ evaluation
+ threshold design
+ fallback behavior
+ drift detection
+ human review
```

Jev doesn't eliminate those engineering problems. In some ways, it makes them **more explicit**. Because once your software starts acting on probabilities, someone has to decide what probability is good enough. Is 0.80 enough? 0.90? 0.99? And should that threshold be different for:

> "send the email"

versus:

> "transfer $50,000"?

Obviously. And that is why I think the most interesting part of Jev is not actually the model. It is the **interface between the model and the program around it**.

So... Is Jev an AI Agent?

Not really. And this distinction matters. An agent usually has some combination of:

```
state
reasoning
planning
memory
tool use
execution
feedback loops
```

Jev is much narrower. TypeSafe describes it as a model that takes **state + typed questions** and returns **typed probabilistic decisions**. So I think a better mental model is:

> **Jev is a decision engine that an agent can use.**

Or, if you want the slightly more dramatic version:

> **Jev is trying to become the part of an AI system that software can actually trust to make a fork in the road.**

The Weird Part Is That This Feels Like Going Backwards

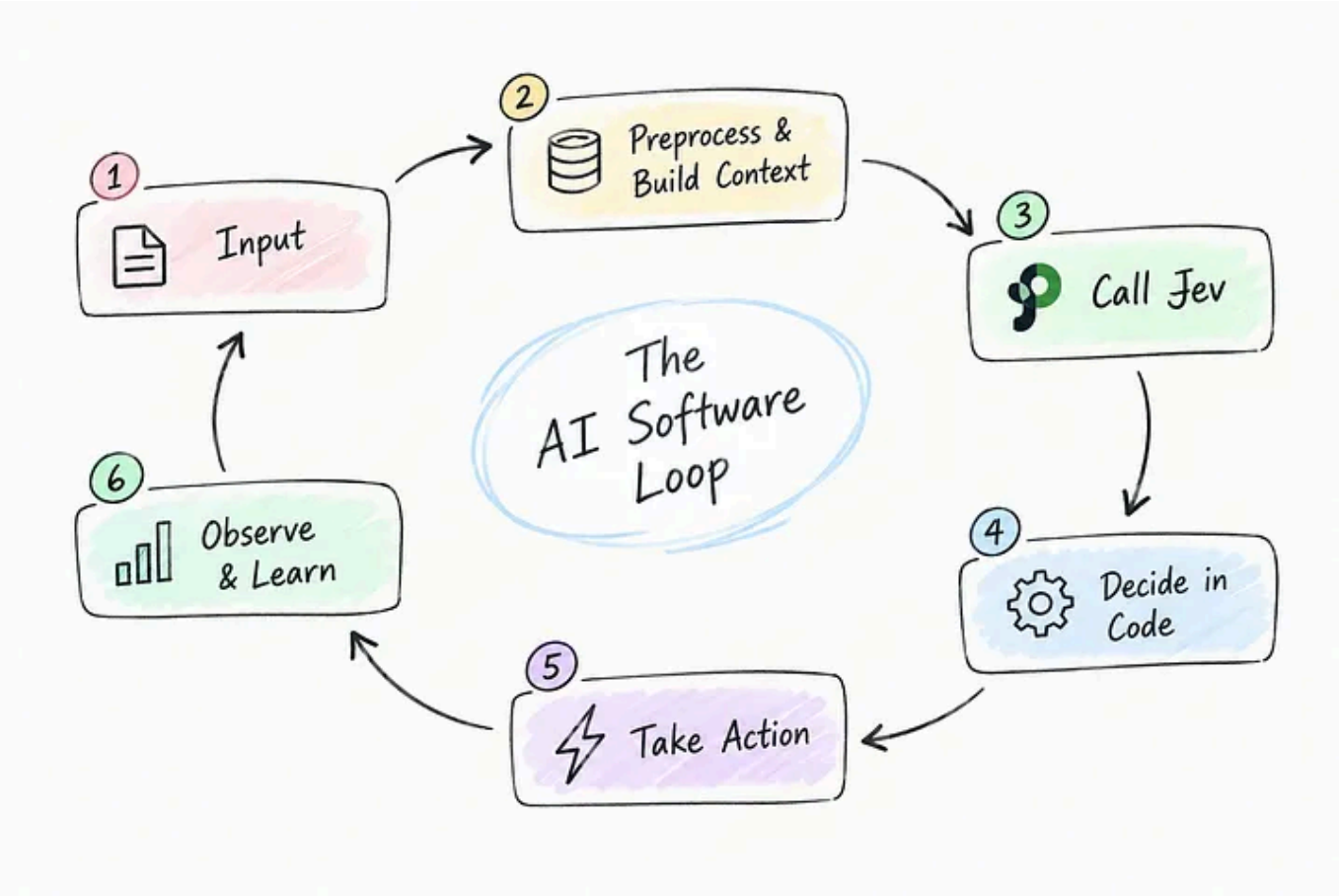


Image by Author — The AI Software Loop

We spent the last few years moving upward in the abstraction stack:



And now TypeSafe is asking a slightly strange question: ***What if the next step is actually going back down?***

Back toward small decision primitives. Back toward interfaces software can depend on. Back toward something that looks a little more like a function call and a little less like a chatbot.

A logic gate doesn't write an essay explaining why its output is `TRUE`. It just gives you the output. The rest of the computer builds something bigger from it.

Jev obviously isn't a transistor or a logic gate. That comparison is an analogy, not a claim about the underlying technology. But conceptually, I think the direction is fascinating:

What if intelligence becomes another primitive software can compose?

Maybe the future of AI isn't everything becoming a chatbot. Maybe some of the most useful models won't talk to us at all.

And Maybe That Is What System One Really Means

But maybe we also need something else. Models that don't talk. Models that don't explain. Models that don't try to be the smartest thing in the room. Models that simply answer the small question the software needs answered. And then get out of the way. That, at least to me, is the part of Jev that is genuinely worth paying attention to.

Few awesome usecases of Jev on Twitter / X

Twitter is flooded with the cool application and demonstration of usecases of Jev, to access Jev you can visit here and apply for waitlist.

1. Self driving car simulation
2. Flight booking with browser automation
3. Resume evaluation against JD by a Stanford CS PhD graduate

4. Use voice to draw on canvas

Give it a search you'll find enormous number of cool stuff people are building with Jev

Typesafe's official release docs

Introducing System One Models & Jev - TypeSafe AI Blog

TypeSafe AI is an AI lab building machine-native intelligence infrastructure for automation, designed to make decisions...

typesafe.ai

> **If you found this reading helpful, don't forget to leave some claps and follow me here on Medium for more breakdowns like this. I write about AI infrastructure, developer tools, and the parts of the stack that quietly don't work the way everyone assumes they do.**

> **You can find all my social handles on my website. I'm most active on LinkedIn and always open to connecting especially if you're building something in the Applied AI , Agentic Systems or AI Safety . If you want to collaborate on an AI project, my DMs are open.**

Artificial Intelligence + | Data Science ✓ | Machine Learning ✓

Programming ✓ | Software Development +



Written by Shreyansh Jain

1K followers · 11 following

Hey, I work at intersection on AI , Computational Algorithms and Maths. I aim to work towards making AI smaller, smarter via better computation and engineering

Follow

Buy me a coffee

Visit me

See more from me





Published in Data Science Collective

960K followers · Last published 4 days ago

Follow

Advice, insights, and ideas from the Medium data science community

Help Status About Careers Press Blog Store Privacy Rules Terms Text to speech